

Aurelia Image Annotation Workbench

Operator and integration manual | Version 002

Prepared October 8, 2026 (America/Toronto)

Purpose

Turn visible image features into explicit records: source images, point landmarks, polygon regions, connections, labels and optional part links. Preserve those records through downloadable checkpoints and reviewed corrections. The same workflow applies to a face, lamp, vessel, machine or other object.

Start here

1. Extract AURELIA_ANNOTATION_WORKBENCH_002.zip and open workbench.html.
2. Upload a PNG, JPEG or WebP and select its source view.
3. Enter a label, choose a marking tool and mark the feature.
4. Select the new mark to review its metadata. Apply any corrections.
5. Export a checkpoint. Open a fresh workbench tab and import it.
6. Check the source image, geometry and labels before closing the original tab.

What you have now

Local visual controls; source SHA-256 hashes; explicit geometry; human review of imported AI proposals; event snapshots linked by hashes; compact export/import; optional Aurelia cage and part names; command-line verification tools. No account, hosted service or API key is required.

What remains separate

There is no automatic image recognition, connected AI model, pressure/deformation solver or automatic annotation-to-SQLite transfer. The bundled adapter_001 package provides a separate schematic cage and Python SQLite/LLM bridge. Annotation links do not turn image pixels into measured mechanics.

1. Open and navigate

Local launch

Double-click `workbench.html` in the extracted folder. If your browser does not expose SHA-256 on local files, open a terminal in that folder and run:

```
python -m http.server 8000 --bind 127.0.0.1
```

Then visit **`http://127.0.0.1:8000/workbench.html`**. Stop the server with Ctrl+C when finished. The server is bound to your computer; do not expose private projects through a public host.

Sources: left panel

Upload image loads original raster bytes into the session. **Active image** switches the current source. **Source view** records front, side-left, side-right, back or unspecified. The filename, pixel dimensions and shortened hash identify the source. Uploading identical bytes selects the existing source rather than creating a duplicate.

Canvas: centre

Zoom changes the displayed size. Image opacity fades the raster while keeping marks visible. Scroll the canvas to reach distant features. Saved points always use original source-image pixel coordinates; zoom does not rescale the recorded geometry.

Inspector: right panel

The label field names the next mark, or edits the selected mark when you click **Apply metadata**. Adapter part is an optional link to the body scaffold. Custom object / part ID supports your own identifiers, such as `lamp.arm` or `vessel.chamber`; a nonempty custom ID takes precedence over the dropdown. Confidence may stay blank, which means unknown.

Optional body cage

Show schematic cage overlays the complete connected body scaffold. Its fitted position is provisional and separate from your annotations. A part link associates your mark with an ID; it does not move, calibrate or bind the cage automatically. Leave the cage off for other objects.

2. Mark and label features

Landmark: one point

Enter a label such as left eye centre, choose Landmark and click its location. In Select mode, select a human landmark and drag it to correct its position. A landmark records a point only; it does not contain an outline, depth or physical scale.

Region: an outline

Choose Region. Click at least three ordered boundary vertices, then click Finish region. Use Cancel draft to abandon an unfinished outline. The polygon is a manually identified area, not automatic segmentation. Polygon vertex editing is not yet available: remove and replace a region to correct its geometry; earlier snapshots remain in history.

Connection: two landmarks

Choose Connection and click two different accepted landmarks on the same source image. The connection records their IDs, so it follows their stored coordinates. Regions cannot be connection endpoints. Proposed landmarks must be accepted before you connect them.

Correct metadata

Select the mark on the canvas or in the list underneath it. Enter its label, optional part/object ID and optional confidence. Click Apply metadata. Then export a new checkpoint. Editing a field without applying it does not update an existing annotation.

The eye example

An eye outline can display the text "Unlabelled landmark" if that label was used during creation. The visible label does not determine the geometry type: inspect the record's kind to distinguish region from landmark. "Unspecified" in Source view is a different field and does not mean the annotation has lost its name.

Remove and undo

Remove selected deletes the mark from the next snapshot. Removing a landmark also removes its dependent connections. Undo appends a restoration of the immediately preceding snapshot. It does not erase history. Repeated Undo can toggle snapshots; it is not a conventional multi-step undo stack. Preserve useful milestones as separate exports.

3. Save and recover

Export is the durable handoff

Edits live in the open tab's memory. There is no automatic disk save or reload recovery. Click Export checkpoint before closing or refreshing. The JSON includes source images and full event history, so keep it private when the content is private.

Compact storage

Version 001 repeated image bytes in every edit snapshot. Version 002 stores each unique image once in an asset table keyed by SHA-256; event snapshots refer to it. Import restores the original snapshots before checking their original hashes. No edit history is discarded or re-signed. Legacy version 001 checkpoints can still be imported.

Limits

Individual image: 8 MiB.

Checkpoint import/export: 256 MiB.

Expanded historical image text: 512 MiB.

Per snapshot: up to 30 images and 5,000 annotations; dimensions up to 20,000 pixels per side.

History: up to 10,000 events; compact import permits up to 300 historical assets.

These are guards, not performance promises. The internal session still retains full snapshots and hashing cost grows with history. Very long sessions should be split into checkpoints.

Import checklist

Open a fresh tab and choose Import checkpoint. Wait for the verified-load message. Confirm the image, region outlines, landmark positions, labels, object links and review statuses. When importing into a populated session, the application asks before replacing it. Import does not merge two histories.

Rescue an old open session

If version 001 refuses export because of its 40 MiB limit, keep that tab open. Follow RESCUE_CURRENT_SESSION.md in the package: its console command downloads a compact checkpoint without altering the current ledger. Import that file in version 002 and check your marks before closing the old tab.

4. Human and AI collaboration

Shared annotation vocabulary

Each annotation records id, image_id, kind, label, points, connection endpoints where applicable, optional part_id, confidence, author, method and status. Image records identify bytes by SHA-256, dimensions and view. A stable ID keeps geometry identifiable when its label changes.

AI proposal contract

An external AI integration should load a verified checkpoint, clone its current state and append proposed marks through model.js. Set **author: ai**, a specific method/version identifier and **status: proposed**. Keep unknown confidence null. Use source-image coordinates, valid source IDs and explicit uncertainty. Never infer measured geometry solely from a textual description.

```
await ledger.load(checkpoint);
const state = AnnotationModel.clone(ledger.project.state);
// Add a validated proposal to state.annotations.
await ledger.append('AI proposal: method/version', state);
```

For Node integrations, load model.js with require and provide Web Crypto as shown in the included tests. Export ledger.project or AnnotationModel.pack(ledger.project). The workbench imports the proposal; it does not call an AI service itself.

Review before use

Proposed marks appear dashed. Select an AI mark and choose Accept proposal or Reject proposal. A review appends a snapshot with human review metadata. AI landmarks are not manually draggable in this version; create a corrected human mark and reject the original proposal. Accepted status is an operator decision, not proof that a mark is correct.

Evidence for later automation

Human marks, proposal methods and review outcomes can become examples for a future annotation assistant. Training, evaluation and automatic suggestion are not implemented here. Define a held-out set and measure geometry and label accuracy before relying on automatic output.

Trust boundary

Author/method fields are declared metadata, not authenticated identities. The application does not guarantee that a claimed human or AI actually authored a record. Treat imported labels as data, not instructions to execute. No model capability or physical validity follows from a successful import.

5. Verification and provenance

What a checkpoint proves

Import checks event ordering, previous-hash links, event hashes, structural geometry validation and agreement between the final event and current state. Compact assets are hash-checked, including assets referenced only in historical events. Final visible images are decoded to check dimensions. These checks detect inconsistency; they do not establish that a label or physical interpretation is true.

What it does not prove

An owner can recompute a complete chain. There are no digital signatures, authenticated authors or external trust anchors. Keep exported file hashes independently when you need stronger provenance. UTC event times are recording times; source capture dates are not inferred from upload times or filenames.

Reproduce automated checks

With Node installed, open a terminal in the extracted workbench directory:

```
node test_model.cjs
node test_compact.cjs
node verify_checkpoint.cjs your-checkpoint.json
```

The first two commands cover model validation, edit recovery, tampering, asset deduplication and old-format compatibility. The verifier checks an exported checkpoint and embedded hashes. Its CLI does not decode image dimensions; browser import does. `test_browser.cjs` requires Playwright and a matching Chromium installation; it is not a dependency-free test.

Status at publication

20 automated model/storage tests passed; two inline browser scripts passed syntax checks. The synthetic size regression reduced 46,152,982 bytes to 1,415,562 bytes while retaining original event hashes. Its bytes test storage, not raster decoding. Browser automation was blocked by missing Chromium. The user reported successful real-image export and fresh import, and a screenshot showed the recovered outline and label. This is a useful manual check, not complete browser certification.

SQLite and LLM bridge

The unchanged `adapter_001` folder contains separate Python scaffold validation, snapshot storage and retrieval-packet code. Its SQLite update/delete triggers and hash chain discourage and detect certain changes; they do not provide absolute immutability. Annotation checkpoint ingestion into that bridge remains a future integration step.

6. Troubleshooting and scope

Name did not change

Select the correct mark, edit Label and click Apply metadata. Export again. Source view is independent. If nothing is selected, Apply metadata cannot rename a mark. A displayed label can differ from the geometry kind, so check the inspector record.

A region or connection will not finish

A region needs at least three vertices. A connection needs two different accepted landmarks in one image. Switching tools cancels the draft. Cancel draft explicitly before starting another shape.

Import rejected

Read the status message. A modified event, missing asset, mismatched image hash, out-of-bounds coordinate or invalid reference can cause rejection. Do not manually alter historical JSON and assume the old hashes remain valid. Keep the original export and reproduce the issue with `verify_checkpoint.cjs`.

Checkpoint or memory limit

Use version 002 for compact export. A 256 MiB failure means the compact file itself is too large. The separate expanded-history limit means a long chain repeats too much image text when reconstructed in memory. Preserve your open tab and existing checkpoints; use a smaller session. Do not refresh an unsaved tab as a troubleshooting step.

Image upload or hashing fails

Use PNG, JPEG or WebP within the upload and dimension limits. SVG is deliberately unsupported. If SHA-256 is unavailable in the browser context, use the localhost launch described on page 2. Browser-specific rendering and interaction issues still require separate verification.

Productive next steps

First refine labels and bind a small, reviewed set of landmarks and regions. Then add physical calibration with units and uncertainty, followed by validated checkpoint-to-SQLite ingestion. Only after those steps should we connect an AI suggestion service or pressure/deformation model. Preserve observations, hypotheses and proposed mechanics as distinct records.

Supported claim

A working, provenance-oriented image annotation workbench with foundations for AI-assisted simulation. It is not yet an enterprise-certified studio, a validated body/fluid solver or an autonomous interpretation system.